

工具调用

使用 Kimi API 完成工具调用 (tool_calls)

📄 复制页面



工具调用，即 `tool_calls`，由函数调用（即 `function_call`）进化而来，在某些特定的语境下，或在阅读一些兼容性代码时，你也可以将工具调用 `tool_calls` 与函数调用 `function_call` 划等号，函数调用 `function_call` 是工具调用 `tool_calls` 的子集。

什么是工具调用 tool_calls

工具调用 `tool_calls` 给予了 Kimi 大模型执行具体动作的能力。Kimi 大模型能进行对话聊天并回答用户提出的问题，这是“说”的能力，而通过工具调用 `tool_calls`，Kimi 大模型也拥有了“做”的能力，借助 `tool_calls`，Kimi 大模型能帮你搜索互联网内容、查询数据库，甚至操作智能家居。

一次工具调用 `tool_calls` 包含了以下若干步骤：

1. 使用 JSON Schema 格式定义工具；
2. 通过 `tools` 参数将定义好的工具提交给 Kimi 大模型，你可以一次性提交多个工具；
3. Kimi 大模型会根据当前聊天的上下文，决定使用哪个或哪几个工具，Kimi 大模型也可以选择不使用工具；
4. Kimi 大模型会将调用工具所需要的参数和信息通过 JSON 格式输出；
5. 使用 Kimi 大模型输出的参数，执行对应的工具，并将工具执行结果提交给 Kimi 大模型；
6. Kimi 大模型根据工具执行结果，给予用户回复；

阅读上述步骤，你可能会产生这样的疑惑：

模型执行工具？既然是我们在执行工具调用，还要 Kimi 大模型干什么？

KIMI 开放平台



我们会用一个实际的工具调用 `tool_calls` 案例来试图向读者讲明白这些问题。

>

通过 `tool_calls` 让 Kimi 大模型拥有联网查询能力

Kimi 大模型的知识来源于它的训练数据，对于一些时效性强的问题，Kimi 大模型无法从自己已有的知识中获取答案，此时，我们希望 Kimi 大模型能自己在互联网上搜索查询最新的知识，并根据这些知识回答我们提出的问题。

定义工具

想象一下，我们自己是如何在网络上找到自己想要的信息的：

1. 我们会先打开搜索引擎，例如百度或必应，在搜索引擎中搜索我们想要的内容，然后浏览搜索结果，根据网站标题和网站简介来决定点击哪个搜索结果；
2. 我们可能会打开一个或多个搜索结果的网页，浏览网页并获取我们需要的知识；

回顾一下我们的动作，我们“使用搜索引擎搜索”和“打开搜索结果对应的网页”，而我们使用的工具是“搜索引擎”和“网页浏览器”，因此，我们需要将动作对应的工具抽象成 JSON Schema 的格式提交给 Kimi 大模型，让 Kimi 大模型也能和人一样使用搜索引擎并浏览网页。

在此之前，让我们先简单介绍一下 JSON Schema 格式：

JSON Schema is a vocabulary that you can use to annotate and validate JSON documents. **JSON Schema** 是一种用于描述 JSON 数据格式的 JSON 文档。

我们定义以下 JSON Schema：

KIMI 开放平台

```
"type": "object",
"properties": {
  "name": {
    "type": "string"
  }
}
```

这个 JSON Schema 定义了一个 JSON Object，这个 JSON Object 中包含了一个名为 `name` 的字段，并且该字段的类型为 `string`，例如：

```
{
  "name": "Hei"
}
```

通过 JSON Schema 来描述我们的工具定义，能让 Kimi 大模型更清晰和直观地知道我们的工具需要哪些参数，以及每个参数的类型和介绍。接下来让我们来定义前文提到的“搜索引擎”和“网页浏览器”这两个工具：

`python` `node.js`

```
{
```

KIMI

开放平台

```
"type": "function", # 约定的字段 type, 目前支持 function 作为值
```

Q

:

```
"function": { # 当 type 为 function 时, 使用 function 字段定义具体的函数内容
```

```
, "name": "search", # 函数的名称, 请使用英文大小写字母、数据加上减号和下划线作为函数名
```

```
"description": ""
```

通过搜索引擎搜索互联网上的内容。

当你的知识无法回答用户提出的问题，或用户请求你进行联网搜索时，调用此工具。请从搜索结果包含网站的标题、网站的地址（URL）以及网站简介。

```
""", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确地理解函数
```

```
"parameters": { # 使用 parameters 字段来定义函数接收的参数
```

```
"type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JSON 对象
```

```
"required": ["query"], # 使用 required 字段告诉 Kimi 大模型哪些参数是必填的
```

```
"properties": { # properties 中是具体的参数定义, 你可以定义多个参数
```

```
"query": { # 在这里, key 是参数名称, value 是参数的具体定义
```

```
"type": "string", # 使用 type 定义参数类型
```

```
"description": ""
```

用户搜索的内容，请从用户的提问或聊天上下文中提取。

```
""", # 使用 description 描述参数以便 Kimi 大模型更好地生成参数
```

```
}
```

```
}
```

```
}
```

```
}
```

```
},
```

```
{
```

```
"type": "function", # 约定的字段 type, 目前支持 function 作为值
```

```
"function": { # 当 type 为 function 时, 使用 function 字段定义具体的函数内容
```

```
"name": "crawl", # 函数的名称, 请使用英文大小写字母、数据加上减号和下划线作为函数名
```

```
"description": ""
```

根据网站地址（URL）获取网页内容。

```
""", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确地理解函数
```

```
"parameters": { # 使用 parameters 字段来定义函数接收的参数
```

```
"type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JSON 对象
```

```
"required": ["url"], # 使用 required 字段告诉 Kimi 大模型哪些参数是必填的
```

```
"properties": { # properties 中是具体的参数定义, 你可以定义多个参数
```

```
"url": { # 在这里, key 是参数名称, value 是参数的具体定义
```

```
"type": "string", # 使用 type 定义参数类型
```

```
"description": ""
```

需要获取内容的网站地址（URL），通常情况下从搜索结果中可以获取网站

KIMI 开放平台



在使用 JSON Schema 定义工具时，我们使用以下固定的格式来定义一个工具：

```
{
  "type": "function",
  "function": {
    "name": "NAME",
    "description": "DESCRIPTION",
    "parameters": {
      "type": "object",
      "properties": {

      }
    }
  }
}
```

其中，`name`、`description`、`parameters.properties` 由工具提供方定义，其中 `description` 描述了工具的具体作用、以及在什么场合需要使用工具，`parameters` 描述了成功调用工具所需要的具体参数，包括参数类型、参数介绍等；最终，Kimi 大模型会根据 JSON Schema 的定义，生成一个满足定义要求的 JSON Object 作为工具调用的参数（arguments）。

注册工具

让我们试试把 `search` 这个工具提交给 Kimi 大模型，看看 Kimi 大模型能否正确调用工具：

python node.js

KIMI 开放平台



```

client = OpenAI(
    api_key="MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台申
    base_url="https://api.moonshot.cn/v1",
)

tools = [
    {
        "type": "function", # 约定的字段 type, 目前支持 function 作为值
        "function": { # 当 type 为 function 时, 使用 function 字段定义具体的函数内容
            "name": "search", # 函数的名称, 请使用英文大小写字母、数据加上减号和下划线作为函
            "description": ""
                通过搜索引擎搜索互联网上的内容。

                当你的知识无法回答用户提出的问题, 或用户请求你进行联网搜索时, 调用此工具。请从
                搜索结果包含网站的标题、网站的地址 (URL) 以及网站简介。

                "", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确地

            "parameters": { # 使用 parameters 字段来定义函数接收的参数
                "type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JSO
                "required": ["query"], # 使用 required 字段告诉 Kimi 大模型哪些参数是必
                "properties": { # properties 中是具体的参数定义, 你可以定义多个参数
                    "query": { # 在这里, key 是参数名称, value 是参数的具体定义
                        "type": "string", # 使用 type 定义参数类型
                        "description": ""
                            用户搜索的内容, 请从用户的提问或聊天上下文中提取。

                            "" # 使用 description 描述参数以便 Kimi 大模型更好地生成参数
                    }
                }
            }
        },
        # {
        #     "type": "function", # 约定的字段 type, 目前支持 function 作为值
        #     "function": { # 当 type 为 function 时, 使用 function 字段定义具体的函数内容
        #         "name": "crawl", # 函数的名称, 请使用英文大小写字母、数据加上减号和下划线作为函
        #         "description": ""
        #             根据网站地址 (URL) 获取网页内容。

        #         "", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确地
    
```

KIMI 开放平台

```
# "required": ["url"], # 使用 required 字段告诉 Kimi 大模型哪些参数是必填
# "properties": { # properties 中是具体的参数定义，你可以定义多个参数
#     "url": { # 在这里，key 是参数名称，value 是参数的具体定义
#         > "type": "string", # 使用 type 定义参数类型
#         "description": ""
#             需要获取内容的网站地址（URL），通常情况下从搜索结果中可以获取网站
#             "" # 使用 description 描述参数以便 Kimi 大模型更好地生成参数
#     }
# }
# }
# }
# }

completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {"role": "system", "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你"},
        {"role": "user", "content": "请联网搜索 Context Caching，并告诉我它是什么。"} #
    ],
    tools=tools, # <-- 我们通过 tools 参数，将定义好的 tools 提交给 Kimi 大模型
)

print(completion.choices[0].model_dump_json(indent=4))
```

当上述代码运行成功时，我们获得 Kimi 大模型的返回内容：

KIMI 开放平台: {

```
"finish_reason": "tool_calls",
"message": {
  "content": "",
  "role": "assistant",
  "tool_calls": [
    {
      "id": "search:0",
      "function": {
        "arguments": "{\\n    \\\"query\\\": \\\"Context Caching\\\"\\n}",
        "name": "search"
      },
      "type": "function",
    }
  ]
}
```

注意看，在这次的回复中，`finish_reason` 的值为 `tool_calls`，这意味着本次请求返回的并不是 Kimi 大模型的回复，而是 Kimi 大模型选择执行工具。你可以通过 `finish_reason` 的值来判断当前 Kimi 大模型的回复是否是一次工具调用 `tool_calls`。

在 `message` 部分，`content` 字段是空值，这是因为当前正在执行 `tool_calls`，模型并没有生成面向用户的回复；同时新增了 `tool_calls` 字段，`tool_calls` 字段是一个列表，其中包含了本次需要调用的所有工具调用信息，这同时也表明了 `tool_calls` 的另一个特性，即：**模型可以一次性选择多个工具进行调用，可以是多个不同的工具，也可以是相同工具使用不同参数进行调用。**`tool_calls` 中的每个元素都代表了一次工具调用，Kimi 大模型会为每次工具调用生成一个唯一的 `id`，通过 `function.name` 字段表明当前执行的工具函数名称，并把执行的参数放置在 `function.arguments` 中，`arguments` 参数是一个合法的被序列化的 JSON Object（额外的，`type` 参数在目前是固定值 `function`）。

接下来，我们应该使用 Kimi 大模型生成的工具调用参数去执行具体的工具。

执行工具

数，在讲述如何执行工具之前，让我们先解答之前提到的问题。

KIMI 开放平台



为什么 Kimi 大模型自己不能执行工具，还要我们根据 Kimi 大模型生成的工具参数“帮”Kimi 大模型执行工具？既然是我们在执行工具调用，还要 Kimi 大模型干什么？

让我们设想一下我们使用 Kimi 大模型的应用场景：我们向用户提供一个基于 Kimi 大模型的人工智能机器人，在这个场景有三个角色：用户、机器人、Kimi 大模型。用户向机器人提问，机器人调用 Kimi 大模型 API，并将 API 的结果返回给用户。当使用 `tool_calls` 时，用户向机器人提问，机器人带着 `tools` 调用 Kimi API，Kimi 大模型返回 `tool_calls` 参数，机器人执行完 `tool_calls`，将结果再次提交给 Kimi API，Kimi 大模型生成返回给用户的消息（`finish_reason=stop`），此时机器人才会把消息返回给用户。在这个过程中，`tool_calls` 的全过程对用户而言都是透明的、隐式的。

回到上述问题，作为用户的我们其实并没有在执行工具调用，也不会直接“看到”工具调用，而是给我们提供服务的机器人在完成工具调用，并将最终 Kimi 大模型生成的回复内容呈现给我们。

让我们以“机器人”的视角来讲解如何执行 Kimi 大模型返回的 `tool_calls`：

python node.js

KIMI 开放平台



```
import httpx
```

```
>
from openai import OpenAI
```

```
client = OpenAI(
    api_key="MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台申
    base_url="https://api.moonshot.cn/v1",
)
```

```
tools = [
    {
        "type": "function", # 约定的字段 type, 目前支持 function 作为值
        "function": { # 当 type 为 function 时, 使用 function 字段定义具体的函数内容
            "name": "search", # 函数的名称, 请使用英文大小写字母、数据加上减号和下划线作为函数名
            "description": """
                通过搜索引擎搜索互联网上的内容。
            """
```

当你的知识无法回答用户提出的问题，或用户请求你进行联网搜索时，调用此工具。请从搜索结果包含网站的标题、网站的地址（URL）以及网站简介。

```
""", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确地理解
        "parameters": { # 使用 parameters 字段来定义函数接收的参数
            "type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JSON 对象
            "required": ["query"], # 使用 required 字段告诉 Kimi 大模型哪些参数是必填的
            "properties": { # properties 中是具体的参数定义, 你可以定义多个参数
                "query": { # 在这里, key 是参数名称, value 是参数的具体定义
                    "type": "string", # 使用 type 定义参数类型
                    "description": """
                        用户搜索的内容, 请从用户的提问或聊天上下文中提取。
                    """ # 使用 description 描述参数以便 Kimi 大模型更好地生成参数
```

```
                }
            }
        }
    },
    {
        "type": "function", # 约定的字段 type, 目前支持 function 作为值
```

KIMI

开放平台



"description": ""

根据网站地址 (URL) 获取网页内容。

""", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确地

> "parameters": { # 使用 parameters 字段来定义函数接收的参数

"type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JSO

"required": ["url"], # 使用 required 字段告诉 Kimi 大模型哪些参数是必填

"properties": { # properties 中是具体的参数定义, 你可以定义多个参数

"url": { # 在这里, key 是参数名称, value 是参数的具体定义

"type": "string", # 使用 type 定义参数类型

"description": ""

需要获取内容的网站地址 (URL), 通常情况下从搜索结果中可以获取网站

"" # 使用 description 描述参数以便 Kimi 大模型更好地生成参数

}

}

}

}

}

]

```
def search_impl(query: str) -> List[Dict[str, Any]]:
```

```
    """
```

search_impl 使用搜索引擎对 query 进行搜索, 目前主流的搜索引擎 (例如 Bing) 都提供了 API
你喜欢的搜索引擎 API 进行调用, 并将返回结果中的网站标题、网站链接、网站简介信息放置在一个 d:

这里只是一个简单的示例, 你可能需要编写一些鉴权、校验、解析的代码。

```
    """
```

```
    r = httpx.get("https://your.search.api", params={"query": query})
```

```
    return r.json()
```

```
def search(arguments: Dict[str, Any]) -> Any:
```

```
    query = arguments["query"]
```

```
    result = search_impl(query)
```

```
    return {"result": result}
```

```
def crawl_impl(url: str) -> str:
```

KIMI 开放平台

这里只是一个简单的示例，在实际的网页抓取过程中，你可能需要编写更多的代码来适配复杂的情况，例如网页内容后，你可以根据自己的需要对网页内容进行清洗，只保留文本或移除不必要的内容（例如广告信息）。

```
"""
>

r = httpx.get(url)
return r.text
```

```
def crawl(arguments: dict) -> str:
    url = arguments["url"]
    content = crawl_impl(url)
    return {"content": content}
```

通过 tool_map 将每个工具名称及其对应的函数进行映射，以便在 Kimi 大模型返回 tool_calls 时能

```
tool_map = {
    "search": search,
    "crawl": crawl,
}
```

```
messages = [
    {"role": "system",
     "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你更擅长中文和英文的对话。你"},
    {"role": "user", "content": "请联网搜索 Context Caching，并告诉我它是什么。"} # 在
]
```

```
finish_reason = None
```

我们的基本流程是，带着用户的问题和 tools 向 Kimi 大模型提问，如果 Kimi 大模型返回了 finish

将执行结果以 role=tool 的 message 的形式重新提交给 Kimi 大模型，Kimi 大模型根据 tool_ca

#

1. 如果 Kimi 大模型认为当前的工具调用结果已经可以回答用户问题，则返回 finish_reason: stop

2. 如果 Kimi 大模型认为当前的工具调用结果无法回答用户问题，需要再次调用工具，我们会继续在循

#

在这个过程中，只有当 finish_reason 为 stop 时，我们才会将结果返回给用户。

```
while finish_reason is None or finish_reason == "tool_calls":
    completion = client.chat.completions.create(
```

KIMI

开放平台

```
tools=tools, # <-- 我们通过 tools 参数，将定义好的 tools 提交给 Kimi 大模型

choice = completion.choices[0]
finish_reason = choice.finish_reason
if finish_reason == "tool_calls": # <-- 判断当前返回内容是否包含 tool_calls
    messages.append(choice.message) # <-- 我们将 Kimi 大模型返回给我们的 assistant
    for tool_call in choice.message.tool_calls: # <-- tool_calls 可能是多个，因此
        tool_call_name = tool_call.function.name
        tool_call_arguments = json.loads(tool_call.function.arguments) # <-- 解析 JSON
        tool_function = tool_map[tool_call_name] # <-- 通过 tool_map 快速找到需要调用的工具
        tool_result = tool_function(tool_call_arguments)

        # 使用函数执行结果构造一个 role=tool 的 message，以此来向模型展示工具调用的结果
        # 注意，我们需要在 message 中提供 tool_call_id 和 name 字段，以便 Kimi 大模型
        # 能正确匹配到对应的 tool_call。
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "name": tool_call_name,
            "content": json.dumps(tool_result), # <-- 我们约定使用字符串格式向 Kimi 大模型返回结果
        })

print(choice.message.content) # <-- 在这里，我们才将模型生成的回复返回给用户
```

我们使用 while 循环来执行包含工具调用在内的代码逻辑，这是因为 Kimi 大模型通常不会只执行一次工具调用，尤其是在联网搜索这个场景，通常，Kimi 大模型会先选择调用 `search` 工具，通过 `search` 工具获取搜索结果后，再调用 `crawl` 工具将搜索结果中的 `url` 转换为具体的网页内容，整体的 messages 结构如下所示：

user: prompt

KIMI 开放平台

```
tool_call(tool_call(name=search, arguments={query: query})
tool: search_result(tool_call_id=tool_call.id, name=search)
assistant: tool_call_1(name=crawl, arguments={url: url_1}), tool_call_2(name=crawl
tool: crawl_content(tool_call_id=tool_call_1.id, name=crawl)
tool: crawl_content(tool_call_id=tool_call_2.id, name=crawl)
assistant: message_content(finish_reason=stop)
```

至此，我们完成了“联网查询”工具调用的全过程，如果你实现了自己的 `search` 和 `crawl` 方法，那么当你向 Kimi 大模型要求联网查询时，它会调用 `search` 和 `crawl` 两个工具，并根据工具调用结果给予你正确的回复。

常见问题及注意事项

关于流式输出

在流式输出模式（stream）下，`tool_calls` 同样适用，但有一些需要额外注意的地方，列举如下：

- 在流式输出的过程中，由于 `finish_reason` 将会在最后的数据块中出现，因此建议使用 `delta.tool_calls` 字段是否存在来判断当前回复是否包含工具调用；
- 在流式输出的过程中，会先输出 `delta.content`，再输出 `delta.tool_calls`，因此你必须等待 `delta.content` 输出完成后，才能判断和识别 `tool_calls`；
- 在流式输出的过程中，我们会在最初的数据块中，指明当前调用 `tool_calls` 的 `tool_call.id` 和 `tool_call.function.name`，在后续的数据块中将只输出 `tool_call.function.arguments`；
- 在流式输出的过程中，如果 Kimi 大模型一次性返回多个 `tool_calls`，那么我们会额外使用一个名为 `index` 的字段来标识当前 `tool_call` 的索引，以便于你能正确拼接 `tool_call.function.arguments` 参数，我们使用流式输出章节中的代码例子（不使用 SDK 的场合）来说明如何操作：

python node.js

```
tools = [
```

KIMI 开放平台



```
"type": "function", # 约定的字段 type, 目前支持 function 作为值
"function": { # 当 type 为 function 时, 使用 function 字段定义具体的函数内容
  "name": "search", # 函数的名称, 请使用英文大小写字母、数据加上减号和下划线作为
  "description": ""
    通过搜索引擎搜索互联网上的内容。
```

当你的知识无法回答用户提出的问题，或用户请求你进行联网搜索时，调用此工具。请从搜索结果包含网站的标题、网站的地址（URL）以及网站简介。

```
""", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确
"parameters": { # 使用 parameters 字段来定义函数接收的参数
  "type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JS
  "required": ["query"], # 使用 required 字段告诉 Kimi 大模型哪些参数是
  "properties": { # properties 中是具体的参数定义, 你可以定义多个参数
    "query": { # 在这里, key 是参数名称, value 是参数的具体定义
      "type": "string", # 使用 type 定义参数类型
      "description": ""
        用户搜索的内容, 请从用户的提问或聊天上下文中提取。
      "" # 使用 description 描述参数以便 Kimi 大模型更好地生成参数
    }
  }
}
```

```
header = {
  "Content-Type": "application/json",
  "Authorization": f"Bearer {os.environ.get('MOONSHOT_API_KEY')}",
}
```

```
data = {
  "model": "kimi-k2.6",
  "messages": [
    {"role": "user", "content": "请联网搜索 Context Caching 技术。"}
  ],
  "stream": True,
  "tools": tools, # <-- 添加工具调用
```

使用 httpx 向 Kimi 大模型发出 chat 请求，并获得响应 r

r = httpx.post("https://api.moonshot.cn/v1/chat/completions",

headers=header,

> json=data)

if r.status_code != 200:

raise Exception(r.text)

data: str

在这里，我们预先构建一个 List，用于存放不同的回复消息，由于我们设置了 n=2，因此我们将 List 初始化为

messages = [{}, {}]

在这里，我们使用了 iter_lines 方法来逐行读取响应体

for line in r.iter_lines():

去除每一行收尾的空格，以便更好地处理数据块

line = line.strip()

接下来我们要处理三种不同的情况：

1. 如果当前行是空行，则表明前一个数据块已接收完毕（即前文提到的，通过两个换行符结束数据块）

2. 如果当前行为非空行，且以 data: 开头，则表明这是一个数据块传输的开始，我们去除 data: 并保留后面的内容

3. 如果当前行为非空行，但不以 data: 开头，则表明当前行仍然归属上一个正在传输的数据块，我们将其追加到上一个数据块中

if len(line) == 0:

chunk = json.loads(data)

通过循环获取每个数据块中所有的 choice，并获取 index 对应的 message 对象

for choice in chunk["choices"]:

index = choice["index"]

message = messages[index]

usage = choice.get("usage")

if usage:

message["usage"] = usage

delta = choice["delta"]

role = delta.get("role")

if role:

message["role"] = role

content = delta.get("content")

if content:


```
else:
```

```
    message["content"] = message["content"] + content
```

```
> # 从这里，我们开始处理 tool_calls
```

```
tool_calls = delta.get("tool_calls") # <-- 先判断数据块中是否包含 tool_c
```

```
if tool_calls:
```

```
    if "tool_calls" not in message:
```

```
        message["tool_calls"] = [] # <-- 如果包含 tool_calls，我们初始化
```

```
    for tool_call in tool_calls:
```

```
        tool_call_index = tool_call["index"] # <-- 获取当前 tool_call
```

```
        if len(message["tool_calls"]) < (
```

```
            tool_call_index + 1): # <-- 根据 index 索引扩充 tool_ca
```

```
            message["tool_calls"].extend([{}] * (tool_call_index + 1 -
```

```
            tool_call_index) # <-- 填充空列表
```

```
        tool_call_object["index"] = tool_call_index
```

```
# 下面的步骤，是根据数据块中的信息填充每个 tool_call 的 id、type、fun
```

```
# 在 function 字段中，又包括 name 和 arguments 字段，arguments 字段
```

```
# 依次补充，如同 delta.content 字段一般。
```

```
tool_call_id = tool_call.get("id")
```

```
if tool_call_id:
```

```
    tool_call_object["id"] = tool_call_id
```

```
tool_call_type = tool_call.get("type")
```

```
if tool_call_type:
```

```
    tool_call_object["type"] = tool_call_type
```

```
tool_call_function = tool_call.get("function")
```

```
if tool_call_function:
```

```
    if "function" not in tool_call_object:
```

```
        tool_call_object["function"] = {}
```

```
    tool_call_function_name = tool_call_function.get("name")
```

```
    if tool_call_function_name:
```

```
        tool_call_object["function"]["name"] = tool_call_function_name
```

```
    tool_call_function_arguments = tool_call_function.get("arguments")
```

```
    if tool_call_function_arguments:
```

```
        if "arguments" not in tool_call_object["function"]:
```

```
            tool_call_object["function"]["arguments"] = tool_call_function_arguments
```

```
        else:
```

KIMI 开放平台

```
message["tool_calls"][tool_call_index] = tool_call_object
```



```
        data = "" # 重置 data
    elif line.startswith("data: "):
        data = line.lstrip("data: ")
```

当数据块内容为 [DONE] 时，则表明所有数据块已发送完毕，可断开网络连接

```
if data == "[DONE]":
    break
```

```
else:
```

```
    data = data + "\n" + line # 我们仍然在追加内容时，为其添加一个换行符，因为这可能是
```

在组装完所有 messages 后，我们分别打印其内容

```
for index, message in enumerate(messages):
    print("index:", index)
    print("message:", json.dumps(message, ensure_ascii=False))
    print("")
```

以下是使用 openai SDK 处理流式输出中的 `tool_calls` 的代码示例：

python **node.js**

```
from openai import OpenAI
```

KIMI 开放平台



```
client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url="https://api.moonshot.cn/v1",
)

tools = [
    {
        "type": "function", # 约定的字段 type, 目前支持 function 作为值
        "function": { # 当 type 为 function 时, 使用 function 字段定义具体的函数内容
            "name": "search", # 函数的名称, 请使用英文大小写字母、数据加上减号和下划线作为
            "description": ""
                通过搜索引擎搜索互联网上的内容。

                当你的知识无法回答用户提出的问题, 或用户请求你进行联网搜索时, 调用此工具。请从
                搜索结果包含网站的标题、网站的地址 (URL) 以及网站简介。

                "", # 函数的介绍, 在这里写上函数的具体作用以及使用场景, 以便 Kimi 大模型能正确
        "parameters": { # 使用 parameters 字段来定义函数接收的参数
            "type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JS
            "required": ["query"], # 使用 required 字段告诉 Kimi 大模型哪些参数是
            "properties": { # properties 中是具体的参数定义, 你可以定义多个参数
                "query": { # 在这里, key 是参数名称, value 是参数的具体定义
                    "type": "string", # 使用 type 定义参数类型
                    "description": ""
                        用户搜索的内容, 请从用户的提问或聊天上下文中提取。
                }
            }
        }
    }
]

completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {"role": "user", "content": "请联网搜索 Context Caching 技术。"}
    ],
```

在这里，我们预先构建一个 List，用于存放不同的回复消息，由于我们设置了 n=2，因此我们将 List 初始化为 messages => [{}, {}]

```
for chunk in completion:
    # 通过循环获取每个数据块中所有的 choice，并获取 index 对应的 message 对象
    for choice in chunk.choices:
        index = choice.index
        message = messages[index]
        delta = choice.delta
        role = delta.role
        if role:
            message["role"] = role
        content = delta.content
        if content:
            if "content" not in message:
                message["content"] = content
            else:
                message["content"] = message["content"] + content

# 从这里，我们开始处理 tool_calls
tool_calls = delta.tool_calls # <-- 先判断数据块中是否包含 tool_calls
if tool_calls:
    if "tool_calls" not in message:
        message["tool_calls"] = [] # <-- 如果包含 tool_calls，我们初始化一个空列表
    for tool_call in tool_calls:
        tool_call_index = tool_call.index # <-- 获取当前 tool_call 的 index
        if len(message["tool_calls"]) < (
            tool_call_index + 1): # <-- 根据 index 索引扩充 tool_calls
            message["tool_calls"].extend([{}] * (tool_call_index + 1 - len(message["tool_calls"])))
        tool_call_object = message["tool_calls"][tool_call_index] # <-- 获取 tool_call_object
        tool_call_object["index"] = tool_call_index

# 下面的步骤，是根据数据块中的信息填充每个 tool_call 的 id、type、function
# 在 function 字段中，又包括 name 和 arguments 字段，arguments 字段会由
# 依次补充，如同 delta.content 字段一般。
```

KIMI 开放平台

```
tool_call_object["id"] = tool_call_id
tool_call_type = tool_call.type
if tool_call_type:
    tool_call_object["type"] = tool_call_type
tool_call_function = tool_call.function
if tool_call_function:
    if "function" not in tool_call_object:
        tool_call_object["function"] = {}
    tool_call_function_name = tool_call_function.name
    if tool_call_function_name:
        tool_call_object["function"]["name"] = tool_call_function.name
    tool_call_function_arguments = tool_call_function.arguments
    if tool_call_function_arguments:
        if "arguments" not in tool_call_object["function"]:
            tool_call_object["function"]["arguments"] = tool_call_function.arguments
        else:
            tool_call_object["function"]["arguments"] = tool_call_function.arguments
    tool_call_object["function"] = tool_call_object["function"]
message["tool_calls"][tool_call_index] = tool_call_object

# 在组装完所有 messages 后，我们分别打印其内容
for index, message in enumerate(messages):
    print("index:", index)
    print("message:", json.dumps(message, ensure_ascii=False))
    print("")
```

关于 `tool_calls` 和 `function_call`

`tool_calls` 是 `function_call` 的进阶版，由于 openai 已将 `function_call` 等参数（例如 `functions`）标记为“已废弃”，因此我们的 API 将不再支持 `function_call`。你可以考虑用 `tool_calls` 代替 `function_call`，相比于 `function_call`，`tool_calls` 有以下几个优点：

- 支持并行调用，Kimi 大模型可以一次返回多个 `tool_calls`，你可以在代码中使用并发的方式同时调用这些 `tool_call` 以减少时间消耗；

的 `function_call`，在一定程度上降低了 Tokens 消耗，

KIMI 开放平台



关于 `content`

在使用工具调用 `tool_calls` 的过程中，你可能会发现，在 `finish_reason=tool_calls` 的情况下，偶尔会出现 `message.content` 字段不为空的情况，通常这里的 `content` 内容是 Kimi 大模型在解释当前需要调用哪些工具和为什么需要调用这些工具。它的意义在于，如果你的工具调用过程耗时很长，或是完成一轮对话需要串行调用多次工具，那么在调用工具前给予用户一段描述性的语句，能减少用户因为等待而产生的焦虑或不满情绪，同时，向用户说明当前调用了哪些工具和为什么调用工具，也有助于用户理解整个工具调用的流程，并及时给予干预和矫正（例如用户认为当前工具选择错误，可以及时终止工具调用，或是在下轮对话中通过提示词矫正模型的工具选择）。

关于 Tokens

`tools` 参数中的内容也会被计算在总 Tokens 中，请确保 `tools`、`messages` 中的 Tokens 总数合计不超过模型的上下文窗口大小。

关于消息布局

在使用工具调用的场景下，我们的消息不再是：

```
system: ...
user: ...
assistant: ...
user: ...
assistant: ...
```

这样排布，而是会变成形似

```
user: ...
```

KIMI 开放平台.



```
tool: ...
```

```
tool: ... ,
```

```
assistant: ...
```

这样的排布，需要注意的是，当 Kimi 大模型生成了 `tool_calls` 时，请确保每一个 `tool_call` 都有对应的 `role=tool` 的 message，并且这条 message 设置了正确的 `tool_call_id`，如果 `role=tool` 的 messages 消息数量与 `tool_calls` 的数量不一致会导致错误；如果 `role=tool` 的 messages 中的 `tool_call_id` 与 `tool_calls` 中的 `tool_call.id` 无法对应也会导致错误。

如果你遇到 `tool_call_id not found` 错误

如果你遇到 `tool_call_id not found` 错误，可能是由于你未将 Kimi API 返回的 `role=assistant` 消息添加到 messages 列表中，正确的消息序列应该看起来像这样：

```
system: ...
```

```
user: ...
```

```
assistant: ... # <-- 也许你并未将这一条 assistant message 添加到 messages 列表中
```

```
tool: ...
```

```
tool: ...
```

```
assistant: ...
```

你可以在每次收到 Kimi API 的返回值后，都执行 `messages.append(message)` 来将 Kimi API 返回的消息添加到消息列表中，以避免出现 `tool_call_id not found` 错误。

注意：添加到 messages 列表中位于 `role=tool` 的 message 之前的 assistant messages，必须完整包含 Kimi API 返回的 `tool_calls` 字段及字段值。我们推荐直接将 Kimi API 返回的 `choice.message` “原封不动”地添加到 messages 列表中，以避免可能产生的错误。

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

KIMI 开放平台



< Batch API 指南

联网搜索工具 >

>

